

Solidity Beginner Handbook

Fundamentals, Keywords, Examples and Mental Models

Mahadi Islam Bhuyan, JU CSE

September 11, 2026

Contents

1	What is Solidity?	4
2	Basic Solidity File	4
3	SPDX License	5
4	Pragma	5
5	Contracts	5
6	Basic Data Types	5
6.1	uint	5
6.2	int	6
6.3	address	6
6.4	bool	6
6.5	string	6
7	State Variables	6
8	Visibility	7
8.1	public	7
8.2	external	7
8.3	internal	7
8.4	private	7
9	Functions and Parameters	8
10	Mapping	8
11	Arrays	8
12	Structs	9
13	Creating a Struct	9

14 Data Locations	10
14.1 storage	10
14.2 memory	10
14.3 calldata	11
15 msg.sender	11
16 msg.value	11
17 block.timestamp	11
18 view	12
19 pure	12
20 payable	12
21 require	12
22 revert	13
23 assert	13
24 Constructor	13
25 Receiving and Sending ETH	13
25.1 Receiving ETH	13
25.2 Important distinction	14
26 Sending ETH with call	14
26.1 payable(msg.sender)	14
26.2 call	14
26.3 success	15
27 Complete Wallet Contract	15
28 How withdraw() Works	16
29 Checks-Effects-Interactions Pattern	16
29.1 1. Checks	16
29.2 2. Effects	17
29.3 3. Interactions	17
30 Why State Must Be Updated Before the External Call	17
31 Important Difference: Accounting vs Actual ETH	17
32 Events	18
33 Modifiers	18
34 Keyword Cheat Sheet	19

35 Core Mental Models	20
35.1 Mapping	20
35.2 Struct	20
35.3 Storage	20
35.4 Memory	20
35.5 Calldata	20
35.6 msg.sender	21
35.7 msg.value	21
35.8 payable	21
35.9 call	21
36 Recommended Learning Order	21

1 What is Solidity?

Solidity is a programming language used to create smart contracts for Ethereum and Ethereum-compatible blockchains.

A smart contract is a program deployed on a blockchain.

A useful mental model is:

```
Smart Contract
|
+-- State / Data
|   +-- Variables
|   +-- Mappings
|   +-- Arrays
|   +-- Structs
|
+-- Functions
|   +-- Change data
|   +-- Read data
|   +-- Receive cryptocurrency
|
+-- Events
|
+-- Modifiers
```

Unlike a normal program running on your computer, important contract state can be stored permanently on the blockchain.

2 Basic Solidity File

A basic Solidity file looks like:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Counter {
    uint256 public count;

    ""
    function increment() public {
        count++;
    }
    ""
}
```

There are four important parts here:

1. SPDX license
2. Compiler version
3. Contract
4. State variable and function

3 SPDX License

```
// SPDX-License-Identifier: MIT
```

This tells development tools which open-source license the source code uses.
It does not affect the actual logic of the contract.

4 Pragma

```
pragma solidity ^0.8.20;
```

This specifies the Solidity compiler version range.

5 Contracts

A contract is similar to a class in object-oriented programming.

```
contract Counter {  
  
}
```

A contract can contain:

- State variables
- Functions
- Structs
- Mappings
- Arrays
- Events
- Modifiers
- Constructors

6 Basic Data Types

6.1 uint

uint is an unsigned integer.

It cannot contain negative numbers.

```
uint256 public balance = 100;
```

For uint256:

$$0 \leq x \leq 2^{256} - 1$$

The maximum value can be obtained with:

```
type(uint256).max
```

6.2 int

An `int` is a signed integer and can contain negative numbers.

```
int8 public temperature = -10;
```

For `int8`:

$$-2^7 \leq x \leq 2^7 - 1$$

Therefore:

$$-128 \leq x \leq 127$$

```
type(int8).min  
type(int8).max
```

6.3 address

An `address` represents a blockchain address.

```
address public owner;
```

6.4 bool

A boolean contains either `true` or `false`.

```
bool public isActive = true;
```

6.5 string

A string stores text.

```
string public name = "Mahadi";
```

7 State Variables

A variable declared directly inside a contract is generally a state variable.

```
contract Example {  
  
    '''  
    uint256 public count;  
    address public owner;  
    string public name;  
    '''  
  
}
```

State variables are stored in persistent blockchain storage.

8 Visibility

Solidity provides four important visibility keywords:

Keyword	Outside	Same Contract	Inherited Contract
Yes	Yes	Yes	Yes
Not directly	Yes via external call	heightexternal	Yes
Yes	No	No	Yes
private		Yes	No

8.1 public

```
function increment() public {  
    count++;  
}
```

A public function can be called externally and internally.

For public state variables, Solidity automatically creates a getter.

```
uint256 public count;
```

The variable can therefore be read externally.

8.2 external

```
function setCount(uint256 value) external {  
    count = value;  
}
```

External functions are intended to be called from outside the contract.

8.3 internal

```
function calculateFee(uint256 amount)  
internal  
pure  
returns (uint256)  
{  
    return amount / 100;  
}
```

Internal functions can be used by the current contract and contracts that inherit from it.

8.4 private

```
function secretCalculation()  
private  
pure  
returns (uint256)  
{  
    return 42;  
}
```

Private functions can only be directly accessed by the contract that defines them.

Important: private does not mean secret on a public blockchain.

9 Functions and Parameters

A function performs an operation.

```
function setCount(uint256 _count) public {  
    count = _count;  
}
```

If someone calls:

```
setCount(100)
```

then:

```
_count = 100  
count = 100
```

10 Mapping

A mapping is a key-value data structure.

```
mapping(address => uint256) public balances;
```

Conceptually:

address	balance
Alice	100
Bob	500
John	250

A value can be stored:

```
balances[msg.sender] = 100;
```

A value can be read:

```
uint256 amount = balances[msg.sender];
```

11 Arrays

A dynamic array can be declared as:

```
uint256[] public numbers;
```

Values can be added using:

```
numbers.push(100);  
numbers.push(200);  
numbers.push(300);
```

Result:

```
[100, 200, 300]
```

Elements are accessed by index:

```
numbers[0]
```

12 Structs

A struct creates a custom data type.

```
struct UserBalance {  
    uint256 balance;  
    uint256 updatedAt;  
}
```

A mapping can store these structs:

```
mapping(address => UserBalance) public users;
```

Conceptually:

```
Alice  
|  
+-- balance: 500  
|  
+-- updatedAt: 123456789
```

13 Creating a Struct

Consider:

```
UserBalance memory userBalance = UserBalance({  
    balance: _newBalance,  
    updatedAt: block.timestamp  
});
```

There are two different uses of `UserBalance`.

The first is the variable's type:

```
UserBalance memory userBalance
```

The second creates a new value of that type:

```
UserBalance({  
    balance: _newBalance,  
    updatedAt: block.timestamp  
})
```

Therefore:

TYPE	LOCATION	VARIABLE	VALUE
UserBalance	memory	userBalance	UserBalance(...)

In English:

Create a temporary variable called `userBalance` of type `UserBalance`, and initialize it with the new balance and the current block timestamp.

14 Data Locations

Solidity has three important data locations:

Location	Permanent?	Modifiable?	Typical Use
Yes	Yes	Blockchain state	storage
Yes	Temporary data/copies	heightmemory	No
External function input	heightcalldata	No	No

14.1 storage

```
UserBalance storage user = users[msg.sender];
```

This creates a reference to the actual stored struct.
Therefore:

```
user.balance = 500;
```

changes the stored blockchain data.
Mental model:

```
Blockchain Storage
^
|
reference
|
user
```

14.2 memory

```
UserBalance memory user = UserBalance({
balance: 100,
updatedAt: block.timestamp
});
```

This creates a temporary copy/value.
It can be modified:

```
user.balance = 500;
```

but modifying the memory variable does not automatically modify blockchain storage.

14.3 calldata

```
function setName(string calldata name)
external
{
}
```

`calldata` is temporary and read-only.
It is commonly used for parameters of external functions.

15 msg.sender

`msg.sender` is the address that called the current function.

```
function setBalance(uint256 amount) public {
balances[msg.sender] = amount;
}
```

If Alice calls the function:

```
msg.sender = Alice
```

Therefore:

```
balances[Alice] = amount
```

16 msg.value

`msg.value` represents the amount of native cryptocurrency sent with the current call.

```
function deposit() public payable {
balances[msg.sender] += msg.value;
}
```

If Alice sends 1 ETH:

```
msg.sender = Alice
msg.value  = 1 ETH
```

17 block.timestamp

`block.timestamp` gives the timestamp associated with the current block.

```
updatedAt = block.timestamp;
```

It is commonly used for recording times and deadlines.
It should not be treated as a perfectly precise clock.

18 view

A view function can read state without modifying it.

```
function balanceOf(address user)
public
view
returns (uint256)
{
return balances[user];
}
```

19 pure

A pure function neither reads nor modifies contract state.

```
function add(uint256 a, uint256 b)
public
pure
returns (uint256)
{
return a + b;
}
```

20 payable

A payable function can receive native cryptocurrency.

```
function deposit() public payable {
balances[msg.sender] += msg.value;
}
```

Without payable, the function cannot normally receive native currency.

21 require

require checks a condition.

```
require(
amount > 0,
"Amount must be greater than zero"
);
```

If the condition is false, the transaction reverts.

Example:

```
require(
balances[msg.sender] >= amount,
"Insufficient balance"
);
```

22 revert

`revert` explicitly stops execution and reverts the transaction.

```
if (amount == 0) {  
  revert("Amount cannot be zero");  
}
```

23 assert

`assert` is generally used for internal invariants that should always be true.

```
assert(total >= balance);
```

A useful beginner distinction:

```
require -> validate conditions / user input  
  
revert  -> explicitly stop execution  
  
assert  -> internal invariant that should never fail
```

24 Constructor

A constructor runs once when the contract is deployed.

```
contract Counter {  
  
  '''  
  address public owner;  
  
  constructor() {  
    owner = msg.sender;  
  }  
  '''  
  
}
```

If Alice deploys the contract:

```
owner = Alice
```

25 Receiving and Sending ETH

A contract can receive native cryptocurrency through a payable function.

25.1 Receiving ETH

```
function deposit() public payable {
balances[msg.sender] += msg.value;
}
```

Suppose Alice sends 1 ETH:

```
Alice
|
| 1 ETH
v
deposit()
|
+-- msg.sender = Alice
+-- msg.value = 1 ETH
|
v
balances[Alice] += 1 ETH
```

The contract now holds the ETH, while the mapping records Alice's internal balance.

25.2 Important distinction

The mapping:

```
balances[Alice] = 1 ETH
```

is only an accounting record.

It does not itself transfer ETH.

The actual contract balance and the internal mapping are separate concepts.

26 Sending ETH with call

To actually send native currency from the contract, one common modern pattern is:

```
(bool success, ) =
payable(msg.sender).call{value: amount}("");

require(success, "Transfer failed");
```

Let's break this apart.

26.1 payable(msg.sender)

```
payable(msg.sender)
```

converts the address into a payable address so that the contract can send native currency to it.

26.2 call

```
.call{value: amount}("")
```

performs a low-level call and sends `amount` of native currency.

26.3 success

The call returns whether the transfer succeeded.

```
(bool success, )
```

The first returned value is stored in `success`.

The second return value is ignored using:

```
-
```

Thus:

```
(bool success, )
```

means:

Store the first returned value in `success` and ignore the second returned value.

27 Complete Wallet Contract

The following is a basic wallet example with complete withdrawal logic.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Wallet {
    ...
    mapping(address => uint256) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function balanceOf(address user)
        public
        view
        returns (uint256)
    {
        return balances[user];
    }

    function withdraw(uint256 amount) public {

        require(
            balances[msg.sender] >= amount,
            "Insufficient balance"
        );

        balances[msg.sender] -= amount;

        (bool success, ) =
            payable(msg.sender).call{value: amount}("");
    }
}
```

```
    require(success, "Transfer failed");
}
'''
}
```

28 How withdraw() Works

Suppose Alice has:

```
balances[Alice] = 5 ETH
```

Alice calls:

```
withdraw(2 ETH)
```

The function executes in this order:

```
1. Check balance
   |
   v
2. Decrease internal balance
   |
   v
3. Send ETH to Alice
   |
   v
4. Check whether transfer succeeded
```

After a successful withdrawal:

```
balances[Alice] = 3 ETH
```

and Alice receives:

```
2 ETH
```

29 Checks-Effects-Interactions Pattern

The withdrawal function follows the important Solidity security pattern:

Checks → Effects → Interactions

29.1 1. Checks

Validate the user's request.

```
require(
balances[msg.sender] >= amount,
"Insufficient balance"
);
```

29.2 2. Effects

Update the contract's internal state.

```
balances[msg.sender] -= amount;
```

29.3 3. Interactions

Interact with an external address or contract.

```
(bool success, ) =  
payable(msg.sender).call{value: amount}("");
```

This ordering is an important defense against reentrancy attacks.

30 Why State Must Be Updated Before the External Call

Consider this safer sequence:

```
Check balance  
|  
v  
Decrease balance  
|  
v  
Send ETH
```

The user's balance has already been reduced before control is given to the recipient.

A dangerous sequence would be:

```
Check balance  
|  
v  
Send ETH  
|  
v  
Decrease balance
```

An external contract receiving ETH could potentially execute code during the interaction and attempt to call `withdraw()` again before its balance is updated.

This is the basic idea behind a reentrancy attack.

31 Important Difference: Accounting vs Actual ETH

These are different:

```
balances[Alice]
```

and:

```
address(this).balance
```

The first is your contract's internal accounting.

The second represents the actual native cryptocurrency held by the contract.

For example:

```
balances[Alice] = 2 ETH

address(this).balance = 5 ETH
```

The contract may have 5 ETH in total while Alice's internal recorded balance is 2 ETH.

32 Events

Events allow a contract to emit information that applications can listen for.

Example:

```
event Deposit(
    address indexed user,
    uint256 amount
);
```

Emit it:

```
emit Deposit(msg.sender, msg.value);
```

A frontend can listen for this event.

A complete deposit function could therefore be:

```
event Deposit(
    address indexed user,
    uint256 amount
);

function deposit() public payable {
    '''
    balances[msg.sender] += msg.value;

    emit Deposit(msg.sender, msg.value);
    '''
}
```

33 Modifiers

A modifier allows reusable conditions to be placed around functions.

Example:

```
address public owner;

modifier onlyOwner() {
    require(
        msg.sender == owner,
        "Not the owner"
    )
}
```

```
);  
-;  
}
```

The *represents wherethemodifiedfunction'scoderuns*.

Example:

```
function changeOwner(address newOwner)  
public  
onlyOwner  
{  
owner = newOwner;  
}
```

Conceptually:

```
changeOwner()  
|  
v  
onlyOwner check  
|  
+-- fail -> revert  
|  
+-- pass  
|  
v  
changeOwner code
```

34 Keyword Cheat Sheet

Keyword	Meaning
contract	Defines a smart contract
uint	Unsigned integer
int	Signed integer
address	Blockchain address
bool	True/false value
string	Text
mapping	Key-value data structure
array	Ordered collection of values
struct	Custom group of related values
public	Accessible externally and internally
external	Intended for external calls
internal	Current and inherited contracts
private	Defining contract only
storage	Persistent blockchain data
memory	Temporary modifiable data
calldata	Temporary read-only input data
view	Reads state without modifying it
pure	Does not read or modify state

payable	Allows native currency to be received
returns	Declares function return values
require	Validates a condition and reverts if false
revert	Explicitly reverts execution
assert	Checks an internal invariant
constructor	Runs once during deployment
modifier	Reusable function condition/logic
event	Emits information for off-chain applications
msg.sender	Address calling the function
msg.value	Native currency sent with the call
block.timestamp	Timestamp associated with the current block
call	Low-level external call, commonly used for ETH transfers

35 Core Mental Models

35.1 Mapping

mapping = key $\rightarrow$ value

Example:

```
address -> uint256
```

35.2 Struct

struct = group related values

Example:

```
User
|
+-- balance
+-- updatedAt
```

35.3 Storage

storage = permanent blockchain state

35.4 Memory

memory = temporary modifiable data

35.5 Calldata

calldata = temporary read-only input

35.6 msg.sender

`msg.sender = who called the function`

35.7 msg.value

`msg.value = native currency sent`

35.8 payable

`payable = allows native currency to be received`

35.9 call

`call = low-level interaction with another address`

36 Recommended Learning Order

After understanding these fundamentals, continue in this order:

1. Events
2. Modifiers
3. Errors and custom errors
4. Inheritance
5. Interfaces
6. Receiving and sending ETH
7. Reentrancy
8. Checks-Effects-Interactions
9. Access control
10. ERC-20 tokens
11. ERC-721 NFTs
12. Testing with Foundry
13. OpenZeppelin
14. Smart contract security

The goal is not to memorize keywords.
Instead, ask:

```
What data do I need?  
|  
v  
Which data type?  
|  
v  
Where should it live?  
|  
v  
Who can access it?  
|  
v  
Can the function modify state?  
|  
v  
Can it receive/send cryptocurrency?  
|  
v  
What checks are required?  
|  
v  
Could an external call create a security problem?
```